

From Symmetry toward Weak Asymmetry: Secure Steganography under the Receiver’s Partial Channel Knowledge

Abstract

Provably secure steganography enables covert communication by disguising secret messages as normal communication content while providing a computational indistinguishability guarantee. Recent work typically assumes that the sender can construct stegotext based on explicit conditional distributions, while modeling the receiver as either having access to the same conditional distributions for decoding, or having no such access at all. In many realistic communication settings, however, the receiver has access to only partial knowledge about the conditional distributions, which is not captured by either assumption. We therefore introduce and study a new setting, called weakly asymmetric steganography, in which the sender has access to the full conditional distributions used for stegotext construction, while the receiver only has partial knowledge about them. We design provably secure steganography schemes under several representative forms of receiver-side knowledge. Specifically, we consider binary alphabets in which the receiver only knows whether a symbol’s probability exceeds a given threshold, as well as general alphabets where the receiver only observes a (possibly partial) ordering of symbols by probability. We further evaluate our constructions in language-model-based text steganography, demonstrating that weakly asymmetric steganography achieves non-trivial embedding capacity under partial knowledge, and suggesting a useful reliability–efficiency tradeoff under mismatched conditional distributions.

1 Introduction

As surveillance and content censorship become increasingly prevalent, the right to free and private communication in cyberspace faces serious threats [10, 18]. In response, a wide range of privacy-enhancing technologies for communication have been studied and deployed. Among them, steganography circumvents censorship by concealing the very existence of communication.

Steganography disguises a secret message (i.e., *hiddentext*) as seemingly normal content. The resulting disguised content

(i.e., *stegotext*) is designed to make it difficult for third-party observers to distinguish it from truly normal content (i.e., *coverttext*). Through stegotext-based communication, the communicating parties can exchange hiddentext while external observers (censors or service providers), even with full access to all communication content, remain unable to determine whether any secret communication is taking place.

To make the above communication truly secure, researchers formally defined the security of steganography schemes as the computational indistinguishability between stegotext and coverttext under the *chosen-hiddentext attack* model (the steganographic version of the *chosen-plaintext attack* model in cryptography). Schemes that satisfy this security definition and correctness (i.e., the ability to correctly recover the hiddentext) are referred to as **provably secure steganography** [11, 12]. Intuitively, this means that for any observer with practical computational capabilities, even if they know what the coverttexts are expected to look like and even know or can choose the hiddentext, they still cannot distinguish the stegotext constructed by the sender from genuine coverttext.

To construct stegotext that is computationally indistinguishable from coverttext, recent mainstream steganography models, such as **symmetric steganography** [13], typically assume that the sender has access to an oracle which, given the current communication history, provides the explicit conditional distribution¹ over the next coverttext symbol. Based on this access, the sender can construct stegotext via a secret-driven sampling algorithm that iteratively selects symbols from the conditional distribution as a function of the hiddentext and updates the communication history until natural termination. The rapid development of AIGC (Artificial Intelligence Generated Content) further supports the practicality of this oracle assumption. In particular, modern autoregressive generative models [8, 14, 17, 19, 22], such as large language models, explicitly output the conditional distribution over the next token given the current context. As a result, such models naturally instantiate the assumed oracle.

¹Since the symbol alphabet is finite, this distribution is typically discrete.

While the above sender-side assumption provides a common foundation, existing provably secure steganography models are differentiated by the assumptions placed on the receiver. In symmetric steganography, the receiver is assumed to have the same oracle access as the sender, enabling decoding based on the explicit conditional distributions. In practice, this assumption is typically realized by having both parties deploy the same generative model. In contrast, **undetectable steganography** [23], also referred to as **asymmetric steganography** [1], assumes that the receiver has no access to such oracle and performs decoding solely based on the received stegotext.

However, the two receiver-side assumptions above and their corresponding steganography schemes fail to capture certain practical scenarios. In such settings, the receiver may not have full access to the oracle described above and therefore cannot obtain the complete conditional distributions, yet may still possess partial information about these distributions—for example, the relative ordering (i.e., rank) of symbols induced by the distribution. In what follows, we refer to such information as the receiver’s *partial channel knowledge*.

For instance, in language-model-based text steganography, such partial channel knowledge naturally arises when the receiver is unable to obtain precise conditional distributions due to limitations in hardware, interface access, or deployment cost. Instead, the receiver may only access restricted APIs that expose limited distributional information, such as top- k candidate tokens or partial logits. Alternatively, when only a sampling interface is available, the receiver may still infer indirect information about the underlying distributions by observing statistical patterns across multiple samples. Moreover, even when the receiver attempts to access the conditional distributions using the same generative model as the sender, differences in devices or inference precision may cause the outputs to match only certain statistical properties, rather than the full distributions.

Under these scenarios, symmetric steganography, which relies on precise distribution access at the receiver, may fail to decode. In contrast, asymmetric steganography allows the receiver to operate without oracle access, but typically achieves only extremely limited embedding capacity, making it unsuitable for efficient covert communication. Furthermore, such schemes completely ignore the partial knowledge already available to the receiver, resulting in a waste of potential communication capability.

To address this gap, we introduce the notion of **Weakly Asymmetric Steganography**, which formally models settings where the sender has full access to the conditional distributions, while the receiver possesses partial but nontrivial channel knowledge related to them. Figure 1 illustrates how our setting compares to existing steganography models.

Weakly asymmetric steganography raises several fundamental questions that have not been studied before. One of the most basic questions is whether, given a fixed form of partial

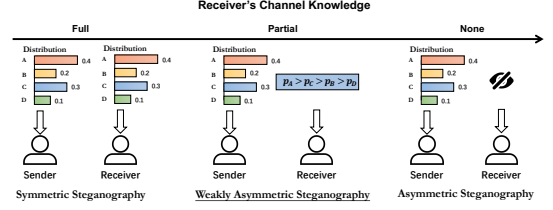


Figure 1: Comparison of symmetric, asymmetric, and weakly asymmetric steganography scenarios.

channel knowledge, there exist weakly asymmetric steganography schemes that can effectively exploit this knowledge. To the best of our knowledge, existing work provides no clear evidence that weakly asymmetric steganography is feasible under any concrete form of partial channel knowledge, beyond the trivial approach of ignoring such knowledge and falling back to fully asymmetric steganography schemes.

In this work, we take a first step toward answering this question by studying and constructing weakly asymmetric steganography schemes under two natural and representative forms of receiver’s partial channel knowledge: threshold-based knowledge in binary alphabets and rank-based channel knowledge in general alphabets. Furthermore, we evaluate the proposed rank-based schemes in the context of language-model-based text steganography, validating their practical feasibility and further examining their robustness under numerical distribution mismatch between the sender and the receiver.

Our main contributions are as follows:

1. We introduce a new formal setting, *weakly asymmetric steganography*, which captures scenarios where the sender has access to the exact numerical values of the channel’s conditional distributions at each step, while the receiver possesses only partial knowledge about these distributions.
2. We initiate the study of weakly asymmetric steganography by designing provably secure schemes under several representative forms of partial channel knowledge. In particular, we present constructions under threshold knowledge in binary alphabets and rank-based knowledge in general alphabets, demonstrating that partial receiver knowledge can be explicitly exploited to achieve meaningful embedding capacity beyond fully asymmetric schemes.
3. We empirically evaluate the proposed rank-based weakly asymmetric schemes in language-model-based text steganography. Our experiments show that weakly asymmetric steganography achieves non-trivial embedding capacity under limited receiver knowledge, and exhibits favorable reliability–efficiency tradeoffs under practical distribution mismatch, compared to both symmetric and fully asymmetric baselines.

Organization

The rest of this paper is organized as follows. Section 2 reviews the necessary background and related work on provably secure steganography. Section 3 introduces the formal model of weakly asymmetric steganography. Section 4 presents concrete constructions of weakly asymmetric steganography schemes under threshold and rank-based channel knowledge. Section 5 reports experimental evaluations of the proposed schemes in language-model-based text steganography, including studies under partial channel knowledge and distribution mismatch.

2 Background and Related Work

Provably Secure Steganography In 2002, Hopper et al. [12] introduced provably secure steganography, formalizing two fundamental requirements: correctness and security. Correctness requires that the receiver’s decoding error be negligible, while security requires that no probabilistic polynomial-time (ppt.) adversary can distinguish stegotexts from coverttexts with non-negligible advantage in the chosen hiddentext attack. This paradigm was initially studied in the *universal steganography* setting, where the sender has access to an oracle that samples from the channel distribution, whereas the receiver has no access to that distribution. Several schemes based on *rejection sampling* have been proposed in this setting [3, 4, 7, 16]. This setting achieves minimal oracle requirements, while existing schemes typically depend on strong and often unrealistic assumptions. For instance, they require the channel to exhibit a positive lower bound on the entropy of its conditional distributions, or assume the existence of suitable unbiased hash functions-conditions that are rarely met in practice [13]. In addition, such schemes generally provide only limited embedding capacity, which further restricts their practical applicability.

Symmetric Steganography with Generative Model Symmetric steganography, formally proposed by Kaptchuk et al. [13], assumes that both the sender and the receiver have access to an oracle that provides the channel distribution. This setting has become one of the most widely adopted paradigms [9, 13, 15, 21] in the study of provably secure steganography, in part because it interfaces naturally with modern generative models [8, 17, 19, 22], and enables practical and efficient stegosystems. Concretely, a generative model can be viewed as a function that maps a history (or prefix) to a conditional distribution over the next symbol, which naturally serves as the channel distribution.

In practice, the most common application scenario of symmetric steganography is text steganography based on language models. Here, the sender transforms a secret message into fluent text by leveraging the shared language model and symmetric steganography scheme. With provable security, the

generated stegotext is computationally indistinguishable from normal language-model outputs. The receiver, equipped with the same language model, can decode the embedded message reliably. The fact that both parties possess the same model provides them with full-distribution access, which enables high embedding capacity. Currently, the symmetric steganography schemes [21] with the highest capacity are approaching entropy, achieving near-theoretical capacity.

However, the correctness of symmetric steganography schemes relies on a fragile consistency requirement: both parties must have access to the same channel distribution, including the receiver, even if it never generates stegotexts. In language-model-based instantiations, this not only demands that the sender and receiver have access to the same model (typically a open-source model), but also that their inference pipelines match closely, since even small differences in device, or numerical precision can change the induced conditional distribution.

Asymmetric Steganography (Undetectable Steganography) *Undetectable watermarking*, proposed by Christ et al. [6], was originally designed for watermarking the outputs of large language models. Its core assumption is that the sender (i.e., the model owner) controls the generation process, while the receiver (i.e., the detector), lacking access to the language model and the prompt, has no knowledge of how the text was produced. The goal is to ensure that watermarked outputs remain computationally indistinguishable from normal text, thereby preserving both linguistic quality and the statistical characteristics of model outputs. Building on this line of work, subsequent research generalized the embedded signal from a single watermark bit to richer information, leading to the notion of *undetectable steganography* [23], which is also referred to as *asymmetric steganography* in later work [2].

Existing schemes [2, 5, 23] typically encode information by biasing the sender’s symbol choice at each generation step based on the secret message. Although the receiver does not know the underlying channel distribution, each symbol has a small amount of mutual information with the secret message, allowing the receiver to recover the message by aggregating many observed symbol and performing hypothesis testing. This design is attractive because the receiver can decode solely from the observed symbols, without requiring access to the channel distribution or maintaining a synchronized channel history. Compared to symmetric steganography, the receiver does not need to run an identical and computationally expensive model as the sender, nor does it need to keep track of past messages. Moreover, it is often robust to certain modifications of the generated stegotext.

However, these advantages come at a cost. As a result, the achievable embedding capacity is significantly constrained, making these schemes less suitable for covert communication scenarios that demand higher throughput. Moreover, these schemes still require careful balancing between the embed-

ding rate and decoding correctness, as decoding is based on hypothesis testing.

3 Definitions

In this section, we introduce the notation used throughout this work, and define weakly asymmetric steganography.

3.1 Notations

Following the formulation of provably secure steganography [11, 13], we use the term *covert channel* to refer to the normal communication channel in the steganographic setting. We write Σ for the alphabet of symbols that may appear in the communication, and denote $\Sigma = \{\sigma_0, \dots, \sigma_{|\Sigma|-1}\}$.

We describe the behavior of a covert channel by a probability distribution, referred to as the *covert channel distribution* (or simply the *channel distribution*). Intuitively, the covert channel distribution describes how normal communication typically continues after any given history. Formally, the covert channel distribution $\mathcal{D}$ is a probability distribution over Σ^* , where Σ^* denotes the set of all finite sequences over Σ . For any history $\mathcal{H} = (h_0, \dots, h_{n-1}) \in \Sigma^*$, $\mathcal{D}$ induces the conditional distribution over Σ of the next symbol in normal communication given $\mathcal{H}$, which we denote by $\mathcal{D}_{\mathcal{H}}$. When unambiguous, we will simply refer to $\mathcal{D}_{\mathcal{H}}$ as the *conditional distribution*, and we will often use D to denote it.

3.2 Knowledge Function & Knowledge

We first formalize the notion of a *knowledge function*, which specifies what information about a distribution is available to a communication entity. Formally, a knowledge function is a mapping

$$f : \mathcal{X} \rightarrow \mathcal{Y},$$

where $\mathcal{X}$ denotes the set of probability distributions over the alphabet Σ , and $\mathcal{Y}$ is the corresponding feature space. For a distribution $D \in \mathcal{X}$, we refer to the value $f(D) \in \mathcal{Y}$ itself as the *knowledge*. Typical examples of knowledge functions include, for instance, the *rank* function $\text{rank}(\cdot)$, which maps a distribution D over Σ to the ordering of symbols by probability.

We say that a communication entity has knowledge f about the channel distribution $\mathcal{D}$ if, for every history $\mathcal{H}$, it has access to the value $f(\mathcal{D}_{\mathcal{H}})$. For brevity, we refer to this access as having knowledge $f(\mathcal{D})$.

We call having knowledge $\mathcal{D}$ itself as having *full channel knowledge*. If f is constant, then having knowledge $f(\mathcal{D})$ provides no nontrivial information about the induced conditional distribution and thus corresponds to having *no channel knowledge*. In this work, we focus on non-degenerate knowledge functions that lie strictly between these two extremes, and we refer to this regime as having *partial channel knowledge*.

3.3 Weakly Asymmetric Steganography

We follow the formal definition of *symmetric steganography* given by Kapatchuk et al. [13] and adapt it to define *weakly asymmetric steganography*, where the receiver has only partial channel knowledge. We state the definition as follows:

Definition 1 (Weakly Asymmetric Steganography). A weakly asymmetric steganography scheme $\Pi_{\mathcal{D},f}$, defined under a covert channel distribution $\mathcal{D}$ and a knowledge function f , is a triple of possibly probabilistic algorithms,

$$\Pi_{\mathcal{D},f} = (\text{KeyGen}_{\mathcal{D}}, \text{Encode}_{\mathcal{D}}, \text{Decode}_{f(\mathcal{D})}).$$

- $\text{KeyGen}_{\mathcal{D}}(1^\lambda)$: generates the secret key sk .
- $\text{Encode}_{\mathcal{D}}(\text{sk}, m, \mathcal{H})$: takes as input a secret key sk , a plaintext message m (also called the *hiddentext*), and the channel history $\mathcal{H}$. It outputs a stegotext $c \in \Sigma^*$.
- $\text{Decode}_{f(\mathcal{D})}(\text{sk}, c, \mathcal{H})$: takes as input a secret key sk , a stegotext c , and the channel history $\mathcal{H}$. It outputs a plaintext message m on success, or the empty string ϵ on failure.

Here, the subscript $\mathcal{D}$ indicates that the algorithm has full channel knowledge, meaning that for any history $\mathcal{H}$, it has access to the exact value of $\mathcal{D}_{\mathcal{H}}$. In contrast, the subscript $f(\mathcal{D})$ indicates that the algorithm has access only to the knowledge specified by $f(\mathcal{D})$, that is, for any history $\mathcal{H}$, it knows only the value $f(\mathcal{D}_{\mathcal{H}})$.

Correctness. A scheme must ensure correctness, i.e., the $\text{Decode}_{f(\mathcal{D})}$ algorithm can recover the embedded message from the stegotext except with negligible probability. Formally, for any secret key $\text{sk} \leftarrow \text{KeyGen}_{\mathcal{D}}(1^\lambda)$, any message m , and any channel history $\mathcal{H}$:

$$\Pr[\text{Decode}_{f(\mathcal{D})}(\text{sk}, \text{Encode}_{\mathcal{D}}(\text{sk}, m, \mathcal{H}), \mathcal{H}) = m] \geq 1 - \text{negl}(\lambda).$$

Security. Security requires that no ppt. adversary, given the full channel distribution $\mathcal{D}$, can distinguish between oracle access to the encoding oracle $\text{Encode}_{\mathcal{D}}(\text{sk}, \cdot, \cdot)$ or a random sampling oracle $O_{\mathcal{D}}(\cdot, \cdot)$ for the channel distribution. Formally, a scheme is secure against *chosen hiddentext attacks*, if for all ppt. adversaries $\mathcal{A}_{\mathcal{D}}$ and $\text{sk} \leftarrow \text{KeyGen}_{\mathcal{D}}(1^\lambda)$, the following inequality holds:

$$\left| \Pr \left[\mathcal{A}_{\mathcal{D}}^{\text{Encode}_{\mathcal{D}}(\text{sk}, \cdot, \cdot)} = 1 \right] - \Pr \left[\mathcal{A}_{\mathcal{D}}^{O_{\mathcal{D}}(\cdot, \cdot)} = 1 \right] \right| \leq \text{negl}(\lambda).$$

Our formulation unifies symmetric and asymmetric steganography through the notion of a knowledge function f . In the symmetric setting, both the sender and the receiver have full channel knowledge. In contrast, a fully asymmetric setting can be modeled by taking f to be a constant function (e.g., $f \equiv 0$), meaning that the receiver has no meaningful channel knowledge. Weakly asymmetric steganography, as studied in this work, captures the intermediate regime where f reveals

some structured information about the channel distribution, while still falling short of full channel knowledge.

For any knowledge function f , every asymmetric steganography scheme is a valid weakly asymmetric steganography scheme, where the receiver can simply ignore $f(\mathcal{D})$. Therefore, the central question is not existence, but non-triviality: whether there exist schemes that meaningfully exploit the receiver's knowledge $f(\mathcal{D})$ to gain a strict advantage over simply ignoring it. We call such schemes non-trivial weakly asymmetric steganography schemes under knowledge f . In this work, we focus on such non-trivial schemes.

Stateful. Consistent with prior work on provably secure steganography [9, 12, 13, 15, 21], the schemes we present later are *stateful*².

4 Schemes under Threshold and Rank Knowledge

A natural question regarding weakly asymmetric steganography is whether this setting has practical significance, or merely constitutes a formal extension of the definitions of symmetric and asymmetric steganography. More concretely, to the best of our knowledge, existing work has not established the existence of non-trivial weakly asymmetric steganography schemes under any form of partial channel knowledge.

This section provides a concrete answer to this question by presenting constructions under two specific forms of partial channel knowledge. We consider *threshold-based knowledge* in the binary alphabet setting, as well as *rank-based knowledge* in general alphabets, demonstrating the existence of non-trivial weakly asymmetric steganography schemes in both cases.

Before introducing the concrete constructions, we first recall a standard technical component used in many symmetric steganography schemes [13, 15], namely a stateful pseudorandom generator.

4.1 Stateful Pseudorandom Generators

Following most classical constructions in symmetric steganography, particularly Meteor [13], our schemes utilize a *stateful* pseudorandom generator (PRG). We follow Meteor's notation for stateful PRGs here.

The stateful PRG consists of two algorithms:

PRG.Setup, PRG.Next.

The setup algorithm generates the initial state material for the PRG and each invocation of PRG.Next outputs fresh pseudorandom bits of sufficiently long fixed length. We interpret this

²There also exist alternative techniques that avoid assuming stateful, such as first sampling a small number of cover symbols and treating them as a nonce [6, 20]. However, such approaches typically rely on assumptions about the channel entropy.

output as a real value in $[0, 1)$ by normalizing the bit string in the natural way. We assume standard PRG security, no ppt. adversary can distinguish oracle access to PRG.Next from oracle access to a truly uniform bit source. Informally, sharing the same PRG key allows the sender and receiver to generate shared pseudorandomness³.

4.2 Binary Alphabet under Threshold Knowledge

For a binary alphabet $\Sigma = \{\sigma_0, \sigma_1\}$, we define a *threshold knowledge function* $\mathbb{1}_\theta(\cdot)$, which indicates whether the probability of symbol σ_0 in the distribution D is below a given threshold θ . Specifically, it outputs:

$$\mathbb{1}_\theta(D) := \mathbb{1}[D(\sigma_0) < \theta] = \begin{cases} 0, & \text{if } D(\sigma_0) \geq \theta, \\ 1, & \text{if } D(\sigma_0) < \theta. \end{cases}$$

This knowledge function reveals the relative relationship between the probability $D(\sigma_0)$ and the threshold θ , and thus also the relative relationship between $D(\sigma_1)$ and $1 - \theta$.

In the binary alphabet case, since a distribution D is completely determined by the single parameter $D(\sigma_0)$, many knowledge functions defined on D , or equivalently on the real value $D(\sigma_0)$, can be expressed in terms of threshold knowledge functions of the form $\mathbb{1}_\theta(\cdot)$. Therefore, in this section we first focus on constructing weakly asymmetric steganography schemes for binary alphabet under the knowledge function $f = \mathbb{1}_\theta(\cdot)$. That is, the sender has full knowledge of the binary channel distribution, while the receiver only knows for each conditional distribution whether the probability of the first symbol exceeds a given threshold θ .

We will first present a construction for the special case $\theta = 0.5$, and then extend it to handle an arbitrary threshold $\theta \in [0, 1)$.

4.2.1 The Special Case $\theta = 0.5$

We begin with the case $\theta = 0.5$, which captures the core ideas of our construction and provides a convenient starting point before extending to arbitrary thresholds. In this setting, the receiver only has access to the value $\mathbb{1}_{0.5}(D)$, rather than the full conditional distribution D .

Equivalently, this means that the receiver's knowledge indicates which symbol is more likely under the binary distribution. Specifically, let $k = \mathbb{1}_{0.5}(D)$. By definition, we have

$$D(\sigma_k) \geq 0.5 \geq D(\sigma_{1-k}),$$

so σ_k is the more likely symbol, while σ_{1-k} is the less likely one.

³Equivalently, the stateful PRG can be replaced by a secure pseudorandom function (PRF) whose input is the channel history $\mathcal{H} \in \Sigma^*$ and whose output lies in $\{0, 1\}^{\text{poly}(\lambda)}$, with the output interpreted as a real-valued random variable in $[0, 1)$.

Next, we describe the sender's Sample algorithm and the receiver's Recovery algorithm. The Sample algorithm specifies how the symbol is sampled based on the secret message for each conditional distribution $\mathcal{D}_{\mathcal{H}}$. The Recovery algorithm specifies how the receiver, given access to $f(\mathcal{D}_{\mathcal{H}})$, recovers the embedded secret information from the received symbol. The full Encode and Decode algorithms are obtained by iterating these steps, as shown in Algorithm 4 and Algorithm 5.

(1) Sample algorithm. Given the current conditional distribution $D = \mathcal{D}_{\mathcal{H}}$, the sender first computes $k = \mathbb{1}_{0.5}(D)$, so that σ_k is the more likely symbol and σ_{1-k} is the less likely one. The sender then draws a fresh pseudorandom value $r \in [0, 1)$ from the shared stateful PRG and lets b denote the first bit of the remaining secret message to be embedded. It then derives

$$r' = \begin{cases} r, & \text{if } b = 0 \\ 1 - r, & \text{if } b = 1 \end{cases}$$

The value r' is used to sample a symbol. Specifically, the unit interval $[0, 1)$ is partitioned according to the distribution D as follows:

$$I_{\sigma_{1-k}} = [0, D(\sigma_{1-k})), \quad I_{\sigma_k} = [D(\sigma_{1-k}), 1),$$

Accordingly, the sender outputs σ_{1-k} if $r' < D(\sigma_{1-k})$, and σ_k otherwise.

In particular, when $b = 0$, we have $r' = r$, and the above procedure reduces to sampling within the partition $(I_{\sigma_{1-k}}, I_{\sigma_k})$ using r . When $b = 1$, we have $r' = 1 - r$, which is equivalent to sampling with r within the swapped partition $(I'_{\sigma_k}, I'_{\sigma_{1-k}})$, where:

$$I'_{\sigma_k} = [0, D(\sigma_k)), \quad I'_{\sigma_{1-k}} = [D(\sigma_k), 1).$$

An illustrative overview is provided in Fig. 2.

If the sampled symbol is σ_{1-k} , the current secret bit b is considered successfully embedded, and the Encode algorithm proceeds to the next bit in the subsequent sampling step. Otherwise, the same secret bit is reused in the next sampling step. The formal description of the Sample algorithm is given in Algorithm 1. This procedure is feasible because the sender has full channel knowledge, in particular the exact values of $D(\sigma_k)$ and $D(\sigma_{1-k})$.

(2) Recovery algorithm. Upon receiving the symbol and the knowledge $k = \mathbb{1}_{0.5}(D)$, the receiver first checks whether the received symbol is σ_{1-k} . If not, then no bit has been embedded in the corresponding sampling step, and hence no further decoding is required at this step.

Once the symbol σ_{1-k} is observed, the decoded bit is determined by

$$\mathbb{1}[r \geq 0.5],$$

where r is the shared pseudorandom value generated by the stateful PRG. Specifically, if $r < 0.5$, then the decoded bit is 0;

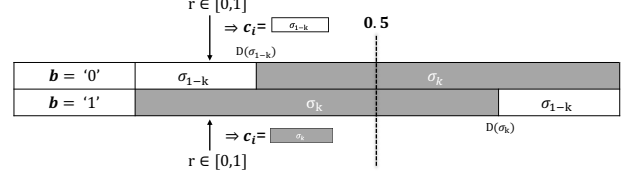


Figure 2: Illustration of the Sample algorithm under different bit b for a given pseudorandom value r . Here, c_i denotes the sampled symbol.

Algorithm 1: $\text{Sample}_{\mathbb{1}_{0.5}}^{\Sigma=\{\sigma_0, \sigma_1\}}(m', \text{PRG}_{\text{sk}}, D)$

Input: Bits m' , Pseudorandom Generator PRG_{sk} , Distribution D

Output: Sampled Symbol c_i , Number of Embedded Bits n_i

$k \leftarrow \mathbb{1}_{0.5}(D), r \leftarrow \text{PRG}_{\text{sk}}.\text{next}(), b = m'[0];$

$r' \leftarrow \begin{cases} r, & \text{if } b = 0 \\ 1 - r, & \text{if } b = 1 \end{cases}$

if $r' < D(\sigma_{1-k})$ **then**

$c_i \leftarrow \sigma_{1-k}, n_i \leftarrow 1;$

else

$c_i \leftarrow \sigma_k, n_i \leftarrow 0;$

return c_i, n_i

otherwise, if $r \geq 0.5$, the decoded bit is 1. A formal description of the Recovery algorithm is given in Algorithm 2.

Notably, the receiver only needs to know the value of k (and hence the identity of σ_{1-k}) and the comparison between the shared randomness r and 0.5 to perform decoding, and does not require access to the exact value of $D(\sigma_{1-k})$.

By iterating the Sample and Recovery algorithms in Algorithm 1 and Algorithm 2, we obtain the corresponding encoding and decoding algorithms $\text{Encode}_{\mathcal{D}}$ and $\text{Decode}_{f(\mathcal{D})}$, as formalized in Algorithm 4 and Algorithm 5. Together with the key generation algorithm $\text{KeyGen}_{\mathcal{D}}$ in Algorithm 3, this yields the following theorem.

Theorem 1. For any covert channel distribution $\mathcal{D}$ over the binary alphabet $\Sigma = \{\sigma_0, \sigma_1\}$, the scheme

$$\Pi_{\mathcal{D}, f} = (\text{KeyGen}_{\mathcal{D}}, \text{Encode}_{\mathcal{D}}, \text{Decode}_{f(\mathcal{D})})$$

defined by Algorithm 3, Algorithm 4, and Algorithm 5, and induced by the Sample and Recovery in Algorithm 1 and Algorithm 2, is a weakly asymmetric steganography scheme under the knowledge function $f = \mathbb{1}_{0.5}(\cdot)$.

Proof sketch of Security. We first provide an informal intuitive proof, where we simplify by assuming that the random numbers r generated by the stateful PRG are independent and uniformly random.

Algorithm 2: $\text{Recovery}_{\Sigma=\{\sigma_0, \sigma_1\}}^{\mathbb{1}_{0.5}}(c_i, \text{PRG}_{\text{sk}}, k)$

Input: Symbol c_i , Pseudorandom Generator PRG_{sk} , Knowledge k

Output: Bit $x \in \{0, 1, \varepsilon\}$

$r \leftarrow \text{PRG}_{\text{sk}}.\text{next}()$;

if $c_i = \sigma_k$ **then**

$x \leftarrow \varepsilon$;

else

if $r < 0.5$ **then**

$x \leftarrow 0$;

else

$x \leftarrow 1$;

return x

Algorithm 3: $\text{KeyGen}_{\mathcal{D}}(1^\lambda)$

Input: Security Parameter 1^λ

Output: Secret Key sk

$\text{sk} \leftarrow \text{PRG}.\text{Setup}(1^\lambda)$;

return sk

Algorithm 4: $\text{Encode}_{\mathcal{D}}(\text{sk}, m, \mathcal{H})$

Input: Secret key sk , Hiddentext m , history $\mathcal{H}$

Output: Stegotext c

$n \leftarrow 0, c \leftarrow \varepsilon$;

while *not the end of* c **do**

$m' = m[n:]$;

$D \leftarrow \mathcal{D}_{\mathcal{H}}$;

$c_i, n_i \leftarrow \text{Sample}_{\mathcal{F}}^{\Sigma}(m', \text{PRG}_{\text{sk}}, D)$;

$c \leftarrow c \| c_i, n \leftarrow n + n_i$;

$\mathcal{H} \leftarrow \mathcal{H} \| c_i$;

return c

Algorithm 5: $\text{Decode}_{f(\mathcal{D})}(\text{sk}, c, \mathcal{H})$

Input: Secret key sk , stegotext c , history $\mathcal{H}$

Output: Plaintext m

$m \leftarrow \varepsilon$;

Let $c = (c_0, c_1, \dots, c_{|c|-1})$;

for $i \leftarrow 0$ **to** $|c| - 1$ **do**

$k \leftarrow f(\mathcal{D}_{\mathcal{H}})$;

$x \leftarrow \text{Recovery}_{\mathcal{F}}^{\Sigma}(c_i, \text{PRG}_{\text{sk}}, k)$;

$m \leftarrow m \| x, \mathcal{H} \leftarrow \mathcal{H} \| c_i$

return m

For any given history $\mathcal{H}$ and the fixed secret bit b , the probability of sampling σ is

$$\Pr_{r \leftarrow \text{Unif}[0,1]}[c_i = \sigma \mid \mathcal{H}, b] = \mathcal{D}_{\mathcal{H}}(\sigma).$$

since the Sample algorithm is based on the value r' , and r' is random regardless of whether $b = 0$ or $b = 1$ (with $r' = r$ when $b = 0$ and $r' = 1 - r$ when $b = 1$). This implies that each execution of the Sample algorithm can be viewed as a random sampling process.

Furthermore, for any two Sample processes, regardless of whether the secret bits are correlated (especially when the same bit is reused), the random values r'_0 and r'_1 used in the two samples are independent because the underlying random values r_0 and r_1 are independent. This further implies that any two executions of the Sample algorithm can be viewed as independent random sampling processes. Therefore, under this assumption (i.e., treating the values r as independent and uniformly random), for any fixed secret message m and secret key sk , a single query to the encoding oracle $\text{Encode}_{\mathcal{D}}$ is indistinguishable from a single query to the channel sampling oracle $\mathcal{O}_{\mathcal{D}}$, since both produce outputs according to the same distribution.

Finally, taking into account the security of the underlying stateful PRG and the fact that the scheme is stateful, a standard hybrid argument yields a formal proof of security.

Proof of Correctness. We show that the Recovery algorithm correctly recovers the bit embedded by the corresponding Sample algorithm.

First, consider the case where the sampled symbol is σ_k , the more likely symbol. In this case, no bit is successfully embedded by the Sample algorithm, and hence no decoding is required. Accordingly, the Recovery algorithm outputs ε , which is correct.

We now consider the case where the sampled symbol is σ_{1-k} , in which exactly one bit is embedded. As illustrated in Fig. 3, if the shared pseudorandom value $r < 0.5$, then sampling σ_{1-k} implies that the embedded bit must be 0. Equivalently,

$$r < 0.5 \wedge c_i = \sigma_{1-k} \implies \text{embedded bit} = 0.$$

Indeed, if the embedded bit were 1, the Sample algorithm would output σ_k when $r < 0.5$, since $D(\sigma_k) \geq 0.5$, contradicting the sampling of σ_{1-k} . Similarly, when $r \geq 0.5$, sampling σ_{1-k} implies that the embedded bit must be 1. That is,

$$r \geq 0.5 \wedge c_i = \sigma_{1-k} \implies \text{embedded bit} = 1.$$

Therefore, in this case the embedded bit can be expressed as

$$\mathbb{1}[r \geq 0.5],$$

which coincides exactly with the decoding rule used by the Recovery algorithm.

Therefore, in all cases, the Recovery algorithm correctly recovers the embedded bit. Since the Encode and Decode algorithms are obtained by iterating the corresponding Sample and Recovery procedures, the correctness of the scheme follows.

	0 ≤ r < 0.5		0.5	0.5 ≤ r ≤ 1	
	0 ≤ r < **	** ≤ r < 0.5		0.5 ≤ r < 1- **	1- ** ≤ r ≤ 1
b = '0'	σ _{1-k}			σ _k	
b = '1'		σ _k			σ _{1-k}

Recovery's view of Sampling process

From the view above, the **Recovery** knows:

When $0 \leq r < 0.5$, and $c_i = \sigma_{1-k}$, **b** must be '0' *Recovery doesn't even need to know **!*
 When $0.5 \leq r \leq 1$, and $c_i = \sigma_{1-k}$, **b** must be '1'

Figure 3: Illustration of the recovery procedure. The marker “**” indicates that the exact value of $D(\sigma_{1-k})$ is unknown to the receiver.

Capacity. We note that the Recovery algorithm can recover exactly 1 bit if and only if the sampled symbol is σ_{1-k} , which occurs with probability $D_{\min} = D[\sigma_{1-k}] \leq 0.5$. Therefore, the capacity (expectation) for this symbol is

$$C_{\mathbb{I}_{0.5}} = 1 \times D_{\min} + 0 \times (1 - D_{\min}) = D_{\min}.$$

4.2.2 General Case $\theta \in [0, 1)$

To generalize the threshold from $\theta = 0.5$ to an arbitrary $\theta \in [0, 1)$, we first reinterpret the Sample and Recovery for the case $\theta = 0.5$.

The sender first applies a bit-controlled transformation to the shared random value r according to the current secret bit b (i.e., choosing to use either r or $1 - r$), and then samples according to the distribution D . Moreover, a secret bit is considered successfully embedded and the sender proceeds to the next bit, if and only if the receiver is able to uniquely decode this bit. On the receiver side, decoding is attempted only upon receiving a symbol whose probability admits a known upper bound (i.e. σ_{1-k}). Furthermore, decoding is performed only when the value r lies in a region that allows the secret bit to be uniquely determined. In the case $\theta = 0.5$, as discussed above, this upper bound equals 0.5, and the corresponding decodable region covers the entire interval $[0, 1)$. From this perspective, extending the construction to a general threshold θ reduces to identifying the range of random values for which the receiver can uniquely decode.

On the sender side, the sampling procedure remains unchanged: given the current secret bit b , the sender applies the same bit-controlled transformation to the shared random value, namely using $r' = r$ if $b = 0$ and $r' = 1 - r$ if $b = 1$, and then samples a symbol according to the distribution D . Consequently, for a general θ , the range of random values for which the receiver can uniquely decode is

$$(0, \min(\theta, 1 - \theta)] \cup (\max(\theta, 1 - \theta), 1]$$

This is because, under the receiver's knowledge $k = \mathbb{I}_\theta(D)$, we have

$$D(\sigma_k) \geq \min(\theta, 1 - \theta) \quad \text{and} \quad D(\sigma_{1-k}) \leq \max(\theta, 1 - \theta),$$

Therefore, when $r < \min(\theta, 1 - \theta)$, observing the symbol σ_{1-k} necessarily implies that it was sampled using $r' = r$, and hence the embedded bit can be uniquely determined to be 0. Similarly, when $r > \max(\theta, 1 - \theta)$, the embedded bit can only be 1. A corresponding intuitive illustration is given in Figure 4.

Algorithm 6: Sample $_{\mathbb{I}_\theta}^{\Sigma=\{\sigma_0, \sigma_1\}}(m', \text{PRG}_{sk}, D)$

Input: Bits m' , Pseudorandom Generator PRG_{sk} , Distribution D

Output: Symbol c_i , Number of Embedded Bits n_i
 $k \leftarrow \mathbb{I}_\theta(D), r \leftarrow \text{PRG}_{sk}.\text{next}(), b = m'[0];$

$r' \leftarrow \begin{cases} r, & \text{if } b = 0 \\ 1 - r, & \text{if } b = 1 \end{cases}$

if $r' < D(\sigma_{1-k})$ **then**

$c_i \leftarrow \sigma_{1-k};$

if $\min(1 - \theta, \theta) < r \leq \max(1 - \theta, \theta)$ **then**

$n_i \leftarrow 0;$

else

$n_i \leftarrow 1;$

else

$c_i \leftarrow \sigma_k, n_i \leftarrow 0;$

return c_i, n_i

Algorithm 7: Recovery $_{\mathbb{I}_\theta(D)}^{\Sigma=\{\sigma_0, \sigma_1\}}(c_i, \text{PRG}_{sk}, \mathcal{H})$

Input: Symbol c_i , Pseudorandom Generator PRG_{sk} , Knowledge k

Output: Bit $x \in \{0, 1, \varepsilon\}$

$r \leftarrow \text{PRG}_{sk}.\text{next}();$

if $c_i = \sigma_k$ **then**

$x \leftarrow \varepsilon;$

else

if $r < \min(1 - \theta, \theta)$ **then**

$x \leftarrow 0;$

else

if $r > \max(1 - \theta, \theta)$ **then**

$x \leftarrow 1;$

else

$x \leftarrow \varepsilon;$

return x

We now directly present the corresponding Sample and Recovery algorithms in Algorithms 6 and 7. Consistent with

the above analysis, it is straightforward to verify that the resulting scheme is both secure and correct, and thus constitutes a weakly asymmetric steganography scheme under knowledge function $f = \mathbb{1}_\theta(\cdot)$. The capacity of this scheme is given by

$$C_{\mathbb{1}_\theta} = \min(D(\sigma_{1-k}), \theta, 1 - \theta).$$

Since $D(\sigma_k) \geq \min(\theta, 1 - \theta)$, we have

$$C_{\mathbb{1}_\theta} = \min(D(\sigma_{1-k}), D(\sigma_k), \theta, 1 - \theta) = \min(D_{\min}, \theta, 1 - \theta),$$

where $D_{\min} := \min\{D(\sigma_0), D(\sigma_1)\}$.

	$\min(\theta, 1 - \theta)$		$\max(\theta, 1 - \theta)$	
	$0 \leq r < \min(\theta, 1 - \theta)$		$\max(\theta, 1 - \theta) \leq r < 1$	
$b = '0'$	σ_{1-k}	?	?	σ_k
$b = '1'$	σ_k	?	?	σ_{1-k}
	$D(\sigma_k) \geq \min(\theta, 1 - \theta)$		$1 - D(\sigma_k) \leq \max(\theta, 1 - \theta)$	

From the view above, the **Recovery** knows:

When $0 \leq r < \min(\theta, 1 - \theta)$, and $c_i = \sigma_{1-k}$, b must be '0'

When $\max(\theta, 1 - \theta) \leq r < 1$, and $c_i = \sigma_k$, b must be '1'

Figure 4: Illustration of the recovery procedure.

4.3 Scheme Design under Rank Knowledge

In this section, we study scheme design under rank-based knowledge over general alphabets $\Sigma = \{\sigma_0, \dots, \sigma_{|\Sigma|-1}\}$.

In this setting, the receiver's knowledge consists of an ordering of symbols induced by the conditional distribution. We begin with the case of *full rank knowledge*, where the receiver has knowledge of the complete ordering over the alphabet. Formally, this is represented by a knowledge function $f = \text{rank}(\cdot)$ which maps a distribution D over the alphabet Σ to a permutation

$$\pi = \text{rank}(D),$$

such that

$$D(\sigma_{\pi(0)}) \geq D(\sigma_{\pi(1)}) \geq \dots \geq D(\sigma_{\pi(|\Sigma|-1)}).$$

We write $\Sigma' = \pi(\Sigma)$ to denote the list Σ ordered according to π , where $\Sigma'[i] = \sigma_{\pi(i)}$.

We then extend the construction to *partial rank knowledge*, where the receiver only observes a restricted ordering over a subset of symbols. In this setting, the receiver's knowledge is represented by

$$\text{PRank}(D) = (\hat{\pi}, \hat{\Sigma}),$$

where $\hat{\Sigma} \subseteq \Sigma$ denotes the subset of symbols whose relative order is known, and $\hat{\pi}$ specifies the ordering over $\hat{\Sigma}$.

4.3.1 Full Rank Knowledge

Our basic idea is to recursively partition the alphabet Σ into two subsets whose total probability masses admit a relative ordering that can be inferred solely from rank knowledge,

ensuring that at each recursive step the receiver can identify the higher-probability subset without accessing exact probability values. Consequently, we can recursively apply the sampling and recovery procedures for threshold knowledge with $\theta = 0.5$, namely $\text{Sample}_{\mathbb{1}_{0.5}}$ and $\text{Recovery}_{\mathbb{1}_{0.5}}$, to perform encoding and decoding over the two subsets. The recursion continues until a single symbol is selected on the sender side, or until all embedded bits have been recovered on the receiver side.

The partitioning rule is simple. For any list Σ' sorted in descending order of probability, we always have

$$\sum_{i \bmod 2=0}^{i < |\Sigma'|} D(\Sigma'[i]) \geq \sum_{i \bmod 2=1}^{i < |\Sigma'|} D(\Sigma'[i]).$$

Therefore, splitting Σ' by parity, as specified in Algorithm 8, induces two subsets whose total probability masses have a known ordering determined purely by rank information.

Algorithm 8: ParitySplit(Σ')

Input: A sorted list Σ'

Output: Two sublists (Σ'_0, Σ'_1)

$\Sigma'_0 \leftarrow \Sigma'[0::2]$;

$\Sigma'_1 \leftarrow \Sigma'[1::2]$;

return (Σ'_0, Σ'_1)

The correctness of the scheme follows from the correctness of the weakly asymmetric scheme under the knowledge function $f = \mathbb{1}_{0.5}$, which ensures correct bit recovery at each recursive step, while rank knowledge and the deterministic partitioning rule guarantee that the decoder's recursion is fully synchronized with the sender's. For security, an intuitive explanation is as follows. At each recursive step, the probability that the $\text{Sample}_{\mathbb{1}_{0.5}}$ selects a given subset is exactly equal to the total probability mass of that subset under the original distribution. As a result, the recursive procedure preserves the original distribution over symbols, and for any history $\mathcal{H}$, secret bits m' and any symbol $\sigma \in \Sigma$, we still have

$$\Pr_{r_0, r_1, \dots \leftarrow \text{Unif}[0,1]} [c_i = \sigma \mid \mathcal{H}, m'] = \mathcal{D}_{\mathcal{H}}(\sigma).$$

The corresponding $\text{Sample}_{\text{rank}}^\Sigma$ and $\text{Recovery}_{\text{rank}}^\Sigma$ are formalized in Algorithm 9 and Algorithm 10.

Capacity. At each recursive step, one bit is embedded if and only if the procedure descends into the lower-probability subset, as dictated by the behavior of the $\text{Sample}_{\mathbb{1}_{0.5}}$ algorithm. Consequently, for any symbol $\sigma_{\pi(i)}$, the total number of embedded bits equals the number of times the lower-probability subset is selected along the recursion, which is exactly the number of ones in the binary representation of i

Let $p_{\pi(i)} = D(\sigma_{\pi(i)})$ denote the probability of the i -th symbol in descending order. The expected embedding capacity is

therefore

$$C_{\text{rank}} = \sum_{i=0}^{|\Sigma|-1} p_{\pi(i)} \cdot \text{count}_1(\text{bin}(i)),$$

where $\text{count}_1(\text{bin}(i))$ denotes the number of ones in the binary representation of i .

This capacity admits the following bounds:

$$1 - p_{\max} \leq C \leq H(D),$$

where $p_{\max} = \max_{\sigma \in \Sigma} D(\sigma)$ and $H(D)$ denotes the entropy of D . The lower bound follows from the fact that every index $i \geq 1$ contains at least one one-bit, and is tight in the binary case. The upper bound holds since for all i ,

$$\text{count}_1(\text{bin}(i)) \leq \log_2(i+1) \leq \log_2 \frac{1}{p_{\pi(i)}},$$

implying that the number of embedded bits per symbol does not exceed its information content.

Algorithm 9: $\text{Sample}_{\text{rank}}^{\Sigma}(m', \text{PRG}_{\text{sk}}, D)$

Input: Bits m' , Pseudorandom Generator PRG_{sk} , Distribution D
Output: Symbol c_i , Number of Embedded Bits n_i
 $n_i \leftarrow 0, \pi \leftarrow \text{rank}(D), \Sigma' \leftarrow \pi(\Sigma) \quad // \text{ descending};$
while $|\Sigma'| > 1$ **do**
 $(\sigma'_0, \sigma'_1) \leftarrow \text{ParitySplit}(\Sigma');$
 $D'(\sigma'_0) \leftarrow \frac{\sum_{x \in \sigma'_0} D(x)}{\sum_{x \in \Sigma'} D(x)};$
 $D'(\sigma'_1) \leftarrow \frac{\sum_{x \in \sigma'_1} D(x)}{\sum_{x \in \Sigma'} D(x)};$
 $\Sigma', b \leftarrow \text{Sample}_{\mathbb{1}_{0.5}}^{\Sigma=\{\sigma'_0, \sigma'_1\}}(m', \text{PRG}_{\text{sk}}, D');$
 $n_i \leftarrow n_i + b;$
 $c_i \leftarrow \Sigma'[0];$
return c_i, n_i

4.3.2 Partial Rank Knowledge

We next consider *partial rank knowledge* $\text{PRank}(\cdot)$, where the receiver only observes an ordering $\hat{\pi}$ over a subset of symbols $\hat{\Sigma} \subseteq \Sigma$.

Our key idea is simple: information is embedded only when the sampled symbol lies in $\hat{\Sigma}$. Otherwise, the sender samples a symbol outside $\hat{\Sigma}$ without embedding any bit. Concretely, the sender first draws a fresh random value $r \leftarrow \text{Unif}[0, 1)$ (which is *not* shared with the receiver) and compares it with the total probability mass

$$p_{\hat{\Sigma}} := \sum_{\sigma \in \hat{\Sigma}} D(\sigma).$$

If $r > p_{\hat{\Sigma}}$, the sender outputs a symbol from $\Sigma \setminus \hat{\Sigma}$ according to the distribution D restricted to and renormalized on

Algorithm 10: $\text{Recovery}_{\text{rank}}^{\Sigma}(c_i, \text{PRG}_{\text{sk}}, \pi)$

Input: Symbol c_i , Pseudorandom Generator PRG_{sk} , Rank Knowledge π
Output: Bits x
 $x \leftarrow \varepsilon, \Sigma' \leftarrow \pi(\Sigma) \quad // \text{ descending};$
while $|\Sigma'| > 1$ **do**
 $(\sigma'_0, \sigma'_1) \leftarrow \text{ParitySplit}(\Sigma');$
 $k = 0; \quad // \Pr(\sigma'_0) \geq \Pr(\sigma'_1)$
 $c'_i \leftarrow \begin{cases} \sigma'_0, & \text{if } c_i \in \sigma'_0; \\ \sigma'_1, & \text{if } c_i \in \sigma'_1; \end{cases}$
 $x \leftarrow x \parallel \text{Recovery}_{\mathbb{1}_{0.5}}^{\Sigma=\{(\sigma'_0, \sigma'_1)\}}(c'_i, \text{PRG}_{\text{sk}}, k);$
return x

$\Sigma \setminus \hat{\Sigma}$, and no bit is embedded. Otherwise, the sender renormalizes D onto $\hat{\Sigma}$ to obtain $D_{\hat{\Sigma}}$, and applies the full-rank scheme $\text{Sample}_{\text{rank}}^{\hat{\Sigma}}$ on the restricted alphabet $\hat{\Sigma}$ to embed bits and sample a symbol.

If the received symbol $c_i \notin \hat{\Sigma}$, the Recovery outputs ε ; otherwise, it decodes using the full rank recovery $\text{Recovery}_{\text{rank}}$ under knowledge $\hat{\pi}$. The corresponding sampling and recovery procedures are given in $\text{Sample}_{\text{PRank}}$ (Algorithm 11) and $\text{Recovery}_{\text{PRank}}$ (Algorithm 12). Correctness and security of the resulting scheme follow directly from those of the full-rank construction. Top- k knowledge arises as a special case by restricting $\hat{\Sigma}$ to the top- k symbols.

Algorithm 11: $\text{Sample}_{\text{PRank}}^{\Sigma}(m', \text{PRG}_{\text{sk}}, D)$

Input: Bits m' , Pseudorandom Generator PRG_{sk} , Distribution D
Output: Symbol c_i , Number of Embedded Bits n_i
 $(\hat{\pi}, \hat{\Sigma}) \leftarrow \text{PRank}(D);$
 $r \leftarrow \text{Unif}[0, 1), p_{\hat{\Sigma}} \leftarrow \sum_{\sigma \in \hat{\Sigma}} D(\sigma);$
if $r > p_{\hat{\Sigma}}$ **then**
 $\text{Sample } c_i \sim \text{Normalize}(D, \hat{\Sigma} \setminus \Sigma);$
 $n_i \leftarrow 0$
else
 $D_{\hat{\Sigma}} \leftarrow \text{Normalize}(D, \hat{\Sigma})$
 $c_i, n_i \leftarrow \text{Sample}_{\text{Rank}}^{\hat{\Sigma}}(m', \text{PRG}_{\text{sk}}, D_{\hat{\Sigma}})$
return c_i, n_i

5 Experiments

In this section, we empirically evaluate weakly asymmetric steganography under rank-based channel knowledge in the setting of language-model-based text steganography.

We conduct two sets of experiments, each highlighting a distinct advantage of weakly asymmetric steganography. First, we evaluate rank-based weakly asymmetric steganog-

Algorithm 12: $\text{Recovery}_{\text{PRank}}^{\hat{\Sigma}}(c_i, \text{PRG}_{\text{sk}}, (\hat{\pi}, \hat{\Sigma}))$

Input: Symbol c_i , Pseudorandom Generator PRG_{sk} ,
Partial Rank Knowledge $(\hat{\pi}, \hat{\Sigma})$

Output: Bits x

if $c_i \notin \hat{\Sigma}$ **then**

return ε

else

$x = \text{Recovery}_{\text{Rank}}^{\hat{\Sigma}}(c_i, \text{PRG}_{\text{sk}}, \hat{\pi})$

return x

raphy when the receiver’s channel knowledge is restricted to rank-based knowledge, such as the ordering of only the top- k tokens. This setting captures a natural usage scenario of weakly asymmetric steganography, where the receiver has access to partial, structure-level channel knowledge but lacks exact probability information.

Second, we investigate rank-based weakly asymmetric steganography under distribution mismatch between the sender and the receiver. In particular, we consider a practically motivated setting where the sender and the receiver run the same language model under different numerical precisions. Although this scenario lies outside our formal model, it reveals an important advantage of the weakly asymmetric perspective: by relying on partial channel knowledge rather than exact probability values, the scheme may admit a favorable reliability–efficiency tradeoff under numerical mismatch.

5.1 Receiver with Partial Channel Knowledge

Setup For the channel distribution, we use two open-source language models: Meta-Llama-3-8B-Instruct⁴ and Qwen2.5-7B-Instruct⁵. The sender accesses the conditional distribution by running the language model directly. To simulate the weakly asymmetric setting, the receiver does not have access to the distribution; instead, we first compute the distribution using the same language model and then extract the corresponding rank-based knowledge, which is provided to the receiver for decoding. We fix the prompt “Tell me something about steganography” as the history. Each stegotext is generated as a response to this, with no length constraint; generation terminates naturally according to the language model.

As baselines, we include asymmetric steganography and random sampling. Symmetric steganography schemes are not included as baselines, since they require full distribution access at the receiver and thus fall outside the weakly asymmetric setting. For asymmetric steganography, we use the Undetectable Steganography scheme [23], based on the official implementation⁶, with the default threshold parameter $t = 2$. We make minor engineering modifications for GPU efficiency,

while preserving the original algorithmic behavior. Random sampling is implemented using `torch.multinomial(num, replacement=True)`; no decoding is performed in this case.

We evaluate both the full-rank scheme and partial-rank schemes. For partial-rank knowledge, the receiver is restricted to the relative ordering of only the top-2, top-5, or top-10 tokens. The pseudorandom number generator (PRG) used in all rank-based weakly asymmetric schemes follows the same construction as in the classical METEOR scheme and is implemented via HMAC-DRBG, with a precision of 52 bits.

All experiments are conducted on a single RTX 5880 Ada GPU with 48GB VRAM. For each experimental configuration, we generate 1,000 responses. Since the history information is fixed, the secret message and cryptographic key are independently re-sampled for each response. The decoding is performed directly at the token level, without converting tokens back to text, in order to avoid ambiguities introduced by text-level representations.

Metrics. *Capacity.* We report the average number of embedded bits per response (**Capacity**), the average empirical entropy per response (**Emp. Entropy**), computed as the sum of token-wise entropies along the Encode process, and the utilization ratio (**Util.**), defined as **Capacity** divided by **Emp. Entropy**. *Time.* We measure the average encoding and decoding time per response (**Enc. Time** and **Dec. Time**), which include both language model inference and steganographic operations. *Other metrics.* We also report the average length of the generated stegotext in number of tokens (**Length**) and the decoding correctness (**Correct.**), measured as the fraction of responses that are decoded correctly.

Results and Analysis. Table 1 summarizes the experimental results under partial rank-based channel knowledge. Rank-based weakly asymmetric steganography exhibits overhead comparable to random sampling. Across both language models, the encoding and decoding time, generated text length, and empirical entropy remain similar to those of random sampling, indicating that the proposed schemes preserve the natural generation behavior of the underlying language models.

At the same time, rank-based weakly asymmetric steganography achieves substantially higher embedding capacity than fully asymmetric steganography. While undetectable steganography embeds only a few bits per response and exhibits limited decoding reliability, the proposed schemes embed tens to hundreds of bits per response with perfect decoding correctness. This demonstrates that partial rank knowledge, even when restricted to top- k ordering information, can be effectively exploited to achieve meaningful embedding capacity beyond fully asymmetric approaches. Overall, these results indicate that weakly asymmetric steganography provides a practical and effective tradeoff between receiver knowledge constraints and usable embedding capacity in restricted-knowledge settings.

⁴<https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct>

⁵<https://huggingface.co/Qwen/Qwen2.5-7B-Instruct>

⁶<https://github.com/OrZamir/steg>

Model	Scheme	Capacity Metrics			Time Metrics		Length	Correct.↑
		Capacity (bit/resp.)↑	Emp. Entropy (bit/resp.)	Util.↑	Enc. Time (s/resp.)↓	Dec. Time (s/resp.)↓		
LLAMA 3 $ \Sigma \approx 128k$	Random Sampling	–	392.681	–	18.798	18.385	497.273	–
	Undect. ($t = 2$) [23]	2.530	388.932	0.65%	21.507	0.143	494.819	61.8%
	Rank (Full)	111.223	391.070	28.44%	20.662	19.115	497.441	100.0%
	Rank (Top10)	95.067	386.317	24.61%	19.115	18.699	492.808	100.0%
	Rank (Top5)	83.374	392.303	21.25%	19.215	18.884	498.172	100.0%
	Rank (Top2)	49.855	390.106	12.78%	18.915	18.753	495.921	100.0%
QWEN 2.5 $ \Sigma \approx 152k$	Random Sampling	–	589.486	–	18.610	18.137	520.575	–
	Undect. ($t = 2$)	2.305	590.922	0.39%	25.384	0.171	519.993	61.6%
	Rank (Full)	170.589	588.066	29.01%	20.486	18.815	518.845	100.0%
	Rank (Top10)	142.931	594.132	24.06%	19.132	18.702	522.571	100.0%
	Rank (Top5)	118.122	597.423	19.77%	19.147	18.808	525.972	100.0%
	Rank (Top2)	63.302	590.385	10.72%	18.748	18.580	521.248	100.0%

Table 1: Experimental results for rank-based weakly asymmetric steganography under partial channel knowledge in language-model-based text steganography.

5.2 Receiver under Distribution Mismatch

We have discussed a basic application of weakly asymmetric steganography, showing that provably secure steganography remains possible even when the receiver has limited channel knowledge. In this section, we study a different but practically relevant scenario: numerical mismatch between the sender’s and receiver’s views of the channel distribution.

In symmetric steganography, decoding correctness typically relies on the sender and receiver obtaining identical and accurate numerical distributions. Even small numerical discrepancies may therefore break decoding correctness, making symmetric schemes highly sensitive to distribution perturbations. In contrast, weakly asymmetric steganography depends only on the receiver’s specified knowledge, rather than exact probability values. As a result, as long as this knowledge remains consistent, decoding can remain reliable even when the underlying distribution changes. From this perspective, weakly asymmetric steganography can be viewed as a steganography model with improved numerical robustness.

To empirically examine this effect, we consider a language-model-based text steganography setting in which the sender and receiver run the same language model under different numerical precisions. Specifically, the sender uses float32 precision, while the receiver performs inference and decoding under float16 precision, inducing a substantial perturbation to the channel distribution. This setting is practically motivated, as float16 inference significantly reduces memory consumption and is commonly adopted under resource constraints.

Setup The experimental setup largely follows that of the previous subsection. We use the same language models, prompt, number of trials, and experimental hardware as before. Here, we focus only on the differences introduced by the distribution mismatch setting.

Unlike the standard setting where the sender and the receiver access identical channel distributions, we introduce a numerical asymmetry between the two parties. Specifically, the sender obtains the channel distribution by running the language model under float32 precision, while the receiver performs inference and decoding using the same model under float16 precision.

In addition to asymmetric and random-sampling baselines, we include several classical symmetric steganography schemes for comparison, including Meteor (16-bit precision) [13], Discop (Huffman) [9], and SparSamp (32-bit) [21]. For fairness, all schemes use pseudorandomness generated via the same HMAC-DRBG construction.

Metrics. Unless otherwise specified, we use the same evaluation metrics as in the previous subsection. Here, we additionally report the following correctness metrics.

Correctness Metrics. **Token Acc.** measures the fraction of tokens for which the decoded bits match the embedded bits. **Resp. Acc.** measures the fraction of responses whose decoded message exactly matches the embedded message. **ED** reports the average edit distance between the decoded message and the embedded message, i.e., the minimum number of edit operations required to transform the decoded message into the original embedded message.

Model	Type	Scheme	Capacity Metrics			Time Metrics		Length	Correct. Metrics		
			Embedd (bit/resp.) $\uparrow$	Entropy (bit/resp.)	Util. $\uparrow$	Enc. Time (s/resp.) $\downarrow$	Dec. Time (s/resp.) $\downarrow$		Token Acc. $\uparrow$	Resp. Acc. $\uparrow$	ED $\downarrow$
LLAMA 3 $ \Sigma \approx 128k$	R.	Random.	–	391.773	–	19.007	9.979	499.007	–	–	–
	S.	Meteor	207.9	385.581	53.9%	19.210	10.566	496.397	99.1%	1.8%	7.439
	S.	Discop(Huff.)	304.6	387.701	78.6%	40.117	31.297	495.700	88.5%	0.9%	40.848
	S.	Sparsamp	353.6	387.782	91.2%	18.942	10.361	496.045	–	0.1%	218.029
	A.	Undect. ($t = 2$)	2.5	388.932	0.65%	21.507	0.143	494.819	–	61.8%	–
	W-A.	Rank (Full)	110.4	387.975	28.5%	20.546	10.503	493.833	99.7%	24.6%	1.910
	W-A.	Rank (Top10)	95.5	388.498	24.6%	19.277	10.368	497.509	99.8%	40.4%	1.140
	W-A.	Rank (Top5)	83.5	389.186	21.4%	19.158	10.355	496.781	99.7%	45.6%	1.364
	W-A.	Rank (Top2)	49.7	391.308	12.7%	18.890	10.285	495.632	99.5%	53.5%	1.859
	R.	Random.	–	589.813	–	18.739	9.708	519.851	–	–	–
QWEN 2.5 $ \Sigma \approx 152k$	S.	Meteor	311.8	577.263	54.0%	18.683	10.206	512.867	98.7%	0.3%	10.535
	S.	Discop(Huff.)	481.3	587.834	81.9%	46.471	36.788	518.430	79.0%	0.0 %	83.220
	S.	Sparsamp	550.1	598.026	92.0%	18.939	10.284	525.455	–	0.0 %	399.092
	A.	Undect. ($t = 2$)	2.3	590.922	0.39%	25.384	0.171	519.993	–	61.6%	–
	W-A.	Rank (Full)	172.6	591.217	29.2%	20.249	10.267	520.008	99.45%	6.7%	3.508
	W-A.	Rank (Top10)	141.9	588.994	24.1%	18.983	10.147	518.817	99.32%	21.5%	2.972
	W-A.	Rank (Top5)	117.3	595.039	19.7%	19.037	10.216	522.867	99.04%	27.7%	3.668
	W-A.	Rank (Top2)	62.6	589.990	10.6%	18.714	10.121	520.096	98.96%	38.1%	3.502
	R.	Random.	–	589.813	–	18.739	9.708	519.851	–	–	–

Table 2: Experimental results under distribution mismatch (float32 sender vs. float16 receiver) for symmetric, asymmetric, and rank-based weakly asymmetric steganography in language-model-based text steganography.

Results and Analysis Table 2 reports the experimental results. This setting is designed to evaluate how different steganography schemes trade off embedding capacity and decoding robustness when the receiver cannot accurately recover the conditional distributions, and is particularly challenging for schemes that rely on precise numerical agreement at the receiver.

While symmetric schemes typically achieve high embedding capacity and entropy utilization, their correctness degrades substantially when precise numerical distributions are unavailable. In particular, several symmetric schemes exhibit low token-level accuracy, large edit distance, and near-zero response-level decoding success, indicating a strong dependence on accurate probability values. Schemes relying on more complex numerical computations, such as SparSamp, are especially sensitive to such distributional deviations.

In contrast, rank-based weakly asymmetric steganography sacrifices part of the embedding capacity in exchange for significantly improved robustness. Weakly asymmetric schemes consistently maintain token-level accuracy close to 99% and substantially lower edit distance under all rank settings. Moreover, their response-level decoding accuracy remains at a usable level. This robustness stems from the fact that decoding

depends only on rank-based channel knowledge, which is generally more stable than exact probability values under numerical perturbations. Given the low edit distance, these results suggest that overall decoding reliability could be further improved with lightweight error-correction mechanisms.

Overall, these results indicate that in realistic settings with distribution mismatch, weakly asymmetric steganography offers a distinct tradeoff compared to symmetric approaches, achieving improved robustness and practical decodability at the cost of reduced embedding capacity.

6 Conclusion

We introduced weakly asymmetric steganography, a new model that captures settings where the receiver possesses partial channel knowledge. We showed that such partial knowledge can be explicitly exploited by constructing provably secure steganography schemes under representative knowledge, including threshold-based knowledge in binary alphabets and rank-based knowledge in general alphabets. Our experimental results in language-model-based text steganography further demonstrate the practical feasibility of this approach under restricted receiver access and distribution mismatch.

Ethical Considerations

This appendix outlines the ethical considerations of our research, following a stakeholder-based analysis.

Stakeholder Analysis and Process Impact Our stakeholders include the research team, future researchers, potential users of steganographic systems, security professionals, and the public. This study is theoretical in nature, with its core work focusing on formalizing security models, designing protocols, and completing security proofs. Its empirical validation uses only publicly available language models and synthetic text, with no involvement of personal privacy data collection, human subjects, or active probing of real-world systems. Therefore, the research process itself does not pose any foreseeable substantive risks to individuals or infrastructure.

Impact of the Publication The *positive impact* of this work lies primarily in providing a more precise security model for the field of provably secure steganography, one that better aligns with practical scenarios. This establishes a more solid theoretical foundation for in-depth analysis of the security boundaries of covert communication, aligning with the principle of beneficence. We also acknowledge the dual-use risk(*negative impact*): like all steganography research, insights could be misused to build more resilient covert channels for illicit purposes. Through transparent theoretical exposition, we ensure the accessibility and verifiability of the academic contributions, and we anticipate that this research will ultimately serve the public interest of enhancing the overall security of digital infrastructure.

Mitigation We have implemented three key measures to mitigate potential misuse. First, we position the work strictly as a theoretical contribution aimed at refining security modeling, not as a deployable tool. Second, we transparently document all security assumptions and limitations of the “weak asymmetry” model to prevent overestimation of its practical utility. Third, all experiments are conducted in a controlled setting with public resources, eliminating procedural harm. This precise formal understanding is necessary for the community to accurately assess the capabilities and limits of steganography under partial knowledge, which is a prerequisite for any informed defensive or policy discussion. We therefore recommend that any future implementation based on this model undergo rigorous, context-specific validation, and that policy discussions account for the formalized spectrum of knowledge asymmetry.

Justification for Research We conducted this research to address a clear gap in steganography theory regarding weak asymmetry. While dual-use risks exist, confining the

work to theory and implementing the above mitigations minimizes them. Conversely, the potential benefits of this research—namely, providing a more rigorous analytical foundation for the field, promoting more effective defensive research, and adhering to the open principles of scientific inquiry—hold significant value. We believe that advancing foundational theory is crucial for safeguarding the long-term interests of communication security. Conducting and submitting such responsible research, centered on models and proofs, is both reasonable and necessary.

Open Science

We comply with the USENIX Security Open Science Policy. All artifacts required to support the evaluation of this paper are available at the anonymized repository below, which will remain accessible throughout the review process: <https://anonymous.4open.science/r/weakly-asy-stego-E7FC>

The repository provides a complete implementation of the experimental pipeline described in Section 5 for our proposed schemes, including the rank-based weakly asymmetric steganography schemes and their empirical instantiation using public language models (e.g., models from Hugging Face). It contains all scripts required to reproduce the reported results. The repository focuses on the implementations of our proposed schemes and baseline schemes are not included. A step-by-step guide in `README.md` describes the environment setup and experimental procedures.

Upon acceptance, we will replace the anonymized link with a permanent, non-anonymous DOI and will keep the artifacts publicly available as required.

References

- [1] Minhao Bai, Jinshuai Yang, Kaiyi Pang, Xu Xin, and Yongfeng Huang. Three-state information hiding: Provably secure asymmetric steganography. *CoRR*, abs/2407.13499, 2024. URL: <https://doi.org/10.48550/arXiv.2407.13499>, arXiv:2407.13499, doi:10.48550/ARXIV.2407.13499.
- [2] Minhao Bai, Jinshuai Yang, Kaiyi Pang, Xin Xu, Zhen Yang, and Yongfeng Huang. Provably robust and secure steganography in asymmetric resource scenario. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 1438–1456. IEEE, 2025.
- [3] Sebastian Berndt and Maciej Liśkiewicz. Provable secure universal steganography of optimal rate: Provably secure steganography does not necessarily imply one-way functions. In *Proceedings of the 4th ACM Workshop on Information Hiding and Multimedia Security*, pages 81–92, 2016.
- [4] Sebastian Berndt and Maciej Liśkiewicz. On the gold standard for security of universal steganography. In *Advances in Cryptology–EUROCRYPT 2018: 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29–May 3, 2018 Proceedings, Part I 37*, pages 29–60. Springer, 2018.
- [5] Miranda Christ and Sam Gunn. Pseudorandom error-correcting codes. In *Annual International Cryptology Conference*, pages 325–347. Springer, 2024.
- [6] Miranda Christ, Sam Gunn, and Or Zamir. Undetectable watermarks for language models. In Shipra Agrawal and Aaron Roth, editors, *The Thirty Seventh Annual Conference on Learning Theory, June 30 - July 3, 2023, Edmonton, Canada*, volume 247 of *Proceedings of Machine Learning Research*, pages 1125–1139. PMLR, 2024. URL: <https://proceedings.mlr.press/v247/christ24a.html>.
- [7] Nenad Dedić, Gene Itkis, Leonid Reyzin, and Scott Russell. Upper and lower bounds on black-box steganography. *Journal of Cryptology*, 22:365–394, 2009.
- [8] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shutong Pan, and S. S. Li. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *CoRR*, abs/2501.12948, 2025. URL: <https://doi.org/10.48550/arXiv.2501.12948>, arXiv:2501.12948, doi:10.48550/ARXIV.2501.12948.
- [9] Jinyang Ding, Kejiang Chen, Yaofei Wang, Na Zhao, Weiming Zhang, and Nenghai Yu. Discop: Provably Secure Steganography in Practice Based on “Distribution Copies”. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 2238–2255. IEEE Computer Society, 2023.
- [10] Phillipa Gill, Masashi Crete-Nishihata, Jakub Dalek, Sharon Goldberg, Adam Senft, and Greg Wiseman. Characterizing web censorship worldwide: Another look at the opennet initiative data. *ACM Transactions on the Web (TWEB)*, 9(1):1–29, 2015.
- [11] Nicholas J Hopper. *Toward a Theory of Steganography*. Carnegie Mellon University, 2004.
- [12] Nicholas J Hopper, John Langford, and Luis Von Ahn. Provably secure steganography. In *Annual International Cryptology Conference*, pages 77–92. Springer, 2002.
- [13] Gabriel Kaptchuk, Tushar M Jois, Matthew Green, and Aviel D Rubin. Meteor: Cryptographically secure steganography for realistic distributions. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 1529–1548, 2021.
- [14] Black Forest Labs. Flux. <https://github.com/black-forest-labs/flux>, 2024.
- [15] Guorui Liao, Jinshuai Yang, Weizhi Shao, and Yongfeng Huang. A framework for designing provably secure steganography. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 6837–6856, 2025.
- [16] Anna Lysyanskaya and Mira Meyerovich. Provably secure steganography with imperfect sampling. In *Public*

Key Cryptography-PKC 2006: 9th International Conference on Theory and Practice in Public-Key Cryptography, New York, NY, USA, April 24-26, 2006. Proceedings 9, pages 123–139. Springer, 2006.

- [17] OpenAI. GPT-4 technical report. *CoRR*, abs/2303.08774, 2023. URL: <https://doi.org/10.48550/arXiv.2303.08774>, arXiv:2303.08774, doi:10.48550/ARXIV.2303.08774.
- [18] Ram Sundara Raman, Prerana Shenoy, Katharina Kohls, and Roya Ensafi. Censored planet: An internet-wide, longitudinal censorship observatory. In *proceedings of the 2020 ACM SIGSAC conference on computer and communications security*, pages 49–66, 2020.
- [19] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models. *CoRR*, abs/2307.09288, 2023. URL: <https://doi.org/10.48550/arXiv.2307.09288>, arXiv:2307.09288, doi:10.48550/ARXIV.2307.09288.
- [20] Tri Van Le. Efficient provably secure public key steganography. *Cryptology ePrint Archive*, 2003.
- [21] Yaofei Wang, Gang Pei, Kejiang Chen, Jinyang Ding, Chao Pan, Weilong Pang, Donghui Hu, and Weiming Zhang. Sparsamp: Efficient provably secure steganography based on sparse sampling. In *34th USENIX Security Symposium (USENIX Security 25)*, 2025.
- [22] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [23] Or Zamir. Undetectable steganography for language models. *Transactions on Machine Learning Research*, 2024.